¿Se puede anticipar el resultado de un partido de selecciones?

Proyecto final · Python para Ciencia de Datos · 2026

Juan Apolo · Alen Cukier

2026-10-11

Tabla de contenidos

1 Introducción **2**

2 Datos **2**
2.1 Fuente . . . 2
2.2 Limpieza . . . 4
2.3 La variable a explicar . . . 4
2.4 Variables construidas: lo que se sabe antes del partido . . . 5
2.5 Muestra final . . . 7

3 Análisis exploratorio **8**
3.1 ¿Cuántos partidos y qué resultados? . . . 8
3.2 ¿Cuántos goles se hacen? . . . 10
3.3 ¿Sirve el Elo? . . . 11
3.4 Uruguay . . . 13

4 Modelado **14**
4.1 Qué comparamos . . . 14
4.2 Modelo de referencia . . . 15
4.3 Árbol de decisión . . . 15
4.4 Random forest . . . 16
4.5 Comparación . . . 17
4.6 Qué variables pesan . . . 18
4.7 Dónde se equivoca . . . 20
4.8 Uruguay en los datos de evaluación . . . 21

5 Comentarios finales **22**

6 Reproducir este trabajo **22**
6.1 Referencias . . . 23

1. Introducción

Un partido de fútbol tiene siempre algo de azar, pero los equipos no llegan al partido en igualdad de condiciones. Antes de que empiece ya sabemos quién viene ganando, quién juega de local y qué tan fuertes han sido históricamente las dos selecciones.

La pregunta que guía el trabajo es: ¿cuánto se puede anticipar el resultado de un partido de selecciones (victoria local, empate o victoria visitante) usando solo información disponible *antes* del pitazo inicial?

Para responderla:

- Partimos de la base de Kaggle con todos los partidos internacionales desde 1872.
- Construimos tres tipos de información previa al partido: la **fuerza histórica** de cada selección (un puntaje Elo que calculamos nosotros), su **forma reciente** (cómo le fue en los últimos 5 partidos) y el **contexto** (si la sede es neutral y si es un amistoso).
- Hacemos un análisis exploratorio para entender la base y ver si esas variables se relacionan con el resultado.
- Comparamos tres modelos de clasificación: una regla de referencia, un árbol de decisión y un random forest.

La variable a explicar es categórica con tres valores (`L`, `E`, `V`), así que el problema es de **clasificación**. No intentamos predecir el marcador exacto.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.metrics import confusion_matrix, classification_report

sns.set_theme(style="whitegrid", palette="colorblind")
pd.set_option("display.max_columns", 20)

AZUL = "#0f3460"
ORDEN = ["L", "E", "V"]
NOMBRES = {"L": "Gana local", "E": "Empate", "V": "Gana visitante"}
```

2. Datos

2.1. Fuente

Usamos **International football results from 1872 to 2026**, de Mart Jürisoo, publicada en Kaggle con licencia CC0. La copia que usamos está en `datos/results.csv` dentro de este repositorio, así que el trabajo corre sin descargar nada.

Cada fila es un partido de selecciones masculinas mayores. Incluye amistosos y competencias oficiales, y también selecciones no afiliadas a FIFA.

```
partidos = pd.read_csv("datos/results.csv", parse_dates=["date"])

print("Filas y columnas:", partidos.shape)
partidos.head()
```

```
Filas y columnas: (49945, 9)
```

	date	home_team	away_team	home_score	away_score	tournament	city	country	neu
0	1872-11-30	Scotland	England	0	0	Friendly	Glasgow	Scotland	Fal
1	1873-03-08	England	Scotland	4	2	Friendly	London	England	Fal
2	1874-03-07	Scotland	England	2	1	Friendly	Glasgow	Scotland	Fal
3	1875-03-06	England	Scotland	2	2	Friendly	London	England	Fal
4	1876-03-04	Scotland	England	3	0	Friendly	Glasgow	Scotland	Fal

```
partidos.dtypes.to_frame("tipo")
```

	tipo
date	datetime64[us]
home_team	str
away_team	str
home_score	int64
away_score	int64
tournament	str
city	str
country	str
neutral	bool

Las variables originales son:

Variable	Tipo	Significado
`date`	fecha	día del partido
`home_team`, `away_team`	categórica	selección local y visitante
`home_score`, `away_score`	numérica	goles de cada equipo al final (incluye alargue, no penales)
`tournament`	categórica	competencia
`city`, `country`	categórica	sede
`neutral`	booleana	si se jugó en cancha neutral

2.2. Limpieza

Revisamos valores faltantes, duplicados y valores imposibles antes de trabajar.

```
partidos.isna().sum().to_frame("faltantes")
```

	faltantes
date	0
home_team	0
away_team	0
home_score	0
away_score	0
tournament	0
city	0
country	0
neutral	0

```
print("Duplicados exactos:", partidos.duplicated().sum())
print("Goles negativos:", ((partidos["home_score"] < 0) | (partidos["away_score"] < 0)).sum())
print("Equipo contra sí mismo:", (partidos["home_team"] == partidos["away_team"]).sum())
print("Rango de fechas:", partidos["date"].min().date(), "a", partidos["date"].max().date())
```

```
Duplicados exactos: 0
Goles negativos: 0
Equipo contra sí mismo: 0
Rango de fechas: 1872-11-30 a 2026-10-06
```

La base está muy limpia: no hay faltantes ni duplicados. Lo único que hacemos es **cortar en el 31/12/2025**, porque 2026 está incompleto y mezclarlo distorsionaría los conteos por año.

```
partidos = partidos[partidos["date"] <= "2025-12-31"].copy()
partidos = partidos.sort_values("date").reset_index(drop=True)
print("Partidos hasta 2025:", len(partidos))
```

```
Partidos hasta 2025: 49144
```

2.3. La variable a explicar

Creamos `resultado` comparando los goles: `L` si ganó el local, `E` si empataron y `V` si ganó el visitante. Agregamos también el total de goles, que usamos en el análisis exploratorio.

```
# np.select elige el primer caso que se cumple; el último valor es el "si no"
partidos["resultado"] = np.select(
    [partidos["home_score"] > partidos["away_score"],
     partidos["home_score"] == partidos["away_score"]],
    ["L", "E"],
    default="V",
)
partidos["total_goles"] = partidos["home_score"] + partidos["away_score"]

partidos["resultado"].value_counts(normalize=True).round(3).to_frame("proporción")
```

resultado	proporción
L	0.490
V	0.283
E	0.227

Una aclaración importante: "local" y "visitante" son los roles que figuran en la base. Cuando la sede es neutral (un Mundial, por ejemplo), el equipo marcado como local **no está jugando en su casa**. Lo tenemos en cuenta más adelante.

2.4. Variables construidas: lo que se sabe antes del partido

No podemos usar los goles del propio partido para predecir su resultado: sería hacer trampa. Lo que sí podemos usar es **lo que pasó antes**. Construimos dos cosas.

Puntaje Elo. Es un número que resume qué tan fuerte viene siendo una selección. Todas arrancan en 1500. Después de cada partido, el ganador le saca puntos al perdedor; cuántos depende de qué tan esperable era el resultado. Ganarle a un equipo mucho más débil suma poco; dar el batacazo suma mucho. Usamos `K = 20` y le damos 60 puntos de ventaja al local cuando la sede no es neutral.

Forma reciente. El promedio de puntos (3 por victoria, 1 por empate, 0 por derrota) y los goles a favor y en contra en los **últimos 5 partidos** de cada selección.

Las dos se calculan recorriendo los partidos en orden cronológico: para cada partido **primero leemos** el Elo y la forma que los equipos traían, y **recién después** los actualizamos con lo que pasó. Así ningún partido usa información de su propio resultado ni del futuro.

```
K = 20            # cuánto se mueve el Elo en cada partido
VENTAJA_LOCAL = 60  # puntos extra para el local si la sede no es neutral
N_FORMA = 5       # cuántos partidos previos mira la "forma reciente"

elo = {}        # equipo -> puntaje Elo actual
historial = {}  # equipo -> lista con sus últimos partidos: (puntos, goles a favor, goles en co

filas = []
```

```python
for p in partidos.itertuples(index=False):
    elo_local = elo.get(p.home_team, 1500.0)
    elo_visita = elo.get(p.away_team, 1500.0)

    prev_local = historial.get(p.home_team, [])
    prev_visita = historial.get(p.away_team, [])

    # Guardamos el estado ANTES de jugar: esto es lo que el modelo puede usar
    filas.append({
        "elo_local": elo_local,
        "elo_visita": elo_visita,
        "n_previos_local": len(prev_local),
        "n_previos_visita": len(prev_visita),
        "pts_local": np.mean([x[0] for x in prev_local[-N_FORMA:]]) if prev_local else np.nan,
        "pts_visita": np.mean([x[0] for x in prev_visita[-N_FORMA:]]) if prev_visita else np.na
        "gf_local": np.mean([x[1] for x in prev_local[-N_FORMA:]]) if prev_local else np.nan,
        "gf_visita": np.mean([x[1] for x in prev_visita[-N_FORMA:]]) if prev_visita else np.nan
        "gc_local": np.mean([x[2] for x in prev_local[-N_FORMA:]]) if prev_local else np.nan,
        "gc_visita": np.mean([x[2] for x in prev_visita[-N_FORMA:]]) if prev_visita else np.nan
    })

    # --- A partir de acá sí miramos el resultado, para actualizar ---
    ventaja = 0 if p.neutral else VENTAJA_LOCAL
    esperado_local = 1 / (1 + 10 ** (-(elo_local - elo_visita + ventaja) / 400))

    if p.home_score > p.away_score:
        real_local, pts_l, pts_v = 1.0, 3, 0
    elif p.home_score == p.away_score:
        real_local, pts_l, pts_v = 0.5, 1, 1
    else:
        real_local, pts_l, pts_v = 0.0, 0, 3

    ajuste = K * (real_local - esperado_local)
    elo[p.home_team] = elo_local + ajuste
    elo[p.away_team] = elo_visita - ajuste

    historial.setdefault(p.home_team, []).append((pts_l, p.home_score, p.away_score))
    historial.setdefault(p.away_team, []).append((pts_v, p.away_score, p.home_score))

previos = pd.DataFrame(filas)
datos = pd.concat([partidos.reset_index(drop=True), previos], axis=1)
print("Elo y forma calculados para", len(datos), "partidos")
```

```
Elo y forma calculados para 49144 partidos
```

Con eso armamos las variables que entran al modelo. Trabajamos con **diferencias** (local menos

visitante) porque lo que importa no es el nivel absoluto sino quién es más fuerte de los dos, y con **promedios** para tener una idea del nivel del partido.

```
datos["elo_dif"] = datos["elo_local"] - datos["elo_visita"]
datos["elo_medio"] = (datos["elo_local"] + datos["elo_visita"]) / 2
datos["pts_dif"] = datos["pts_local"] - datos["pts_visita"]
datos["gf_dif"] = datos["gf_local"] - datos["gf_visita"]
datos["gc_dif"] = datos["gc_local"] - datos["gc_visita"]
datos["sede_neutral"] = datos["neutral"].astype(int)
datos["es_amistoso"] = (datos["tournament"] == "Friendly").astype(int)

VARIABLES = ["elo_dif", "elo_medio", "pts_dif", "gf_dif", "gc_dif",
             "sede_neutral", "es_amistoso"]

datos[VARIABLES].describe().round(2)
```

	elo_dif	elo_medio	pts_dif	gf_dif	gc_dif	sede_neutral	es_amistoso
count	49144.00	49144.00	48842.00	48842.00	48842.00	49144.00	49144.00
mean	7.41	1547.66	0.03	0.03	-0.04	0.26	0.37
std	159.75	117.89	0.98	1.13	1.31	0.44	0.48
min	-853.32	1090.74	-3.00	-14.40	-19.60	0.00	0.00
25 %	-90.29	1473.41	-0.60	-0.60	-0.60	0.00	0.00
50 %	7.31	1540.65	0.00	0.00	0.00	0.00	0.00
75 %	106.60	1624.09	0.60	0.60	0.60	1.00	1.00
max	819.91	1973.97	3.00	16.00	20.00	1.00	1.00

2.5. Muestra final

Nos quedamos con los partidos **desde 1990** y exigimos que las dos selecciones tengan al menos 10 partidos previos. Sin esa historia, el Elo todavía vale 1500 y la forma reciente no significa nada.

```
muestra = datos[
    (datos["date"] >= "1990-01-01")
    & (datos["n_previos_local"] >= 10)
    & (datos["n_previos_visita"] >= 10)
].copy()

print("Partidos en la muestra:", len(muestra))
print("Desde", muestra["date"].min().date(), "hasta", muestra["date"].max().date())
print("Selecciones distintas:", pd.concat([muestra["home_team"], muestra["away_team"]]).nunique
```

```
Partidos en la muestra: 31177
Desde 1990-01-12 hasta 2025-12-31
Selecciones distintas: 275
```

3. Análisis exploratorio

Las preguntas que nos hacemos acá son cuatro:

1. ¿Cuántos partidos hay y cómo se reparten los resultados?
2. ¿De verdad conviene ser local?
3. ¿Cuántos goles se hacen por partido?
4. ¿La diferencia de Elo se relaciona con el resultado?

> Todo el análisis exploratorio usa **solo los datos de entrenamiento** (hasta 2018). No miramos los años que después usamos para evaluar, para no dejarnos influir por ellos al elegir el modelo.

```
train = muestra[muestra["date"] <= "2018-12-31"].copy()
test = muestra[muestra["date"] >= "2019-01-01"].copy()

print(f"Entrenamiento: {len(train):>5} partidos ({train['date'].min().date()} a {train['date'].
print(f"Evaluación:    {len(test):>5} partidos ({test['date'].min().date()} a {test['date'].max
```

```
Entrenamiento: 24338 partidos (1990-01-12 a 2018-12-31)
Evaluación:     6839 partidos (2019-01-02 a 2025-12-31)
```

Partimos **por fecha y no al azar**. Si mezcláramos los años, el modelo podría entrenar con partidos de 2020 y predecir partidos de 2010, que es justamente lo contrario de lo que queremos: anticipar lo que todavía no pasó.

3.1. ¿Cuántos partidos y qué resultados?

```
train["anio"] = train["date"].dt.year

fig, ax = plt.subplots(figsize=(7, 3.2), layout="constrained")
train["anio"].value_counts().sort_index().plot(ax=ax, color=AZUL, linewidth=2)
ax.set(xlabel="Año", ylabel="Partidos")
plt.show()
```

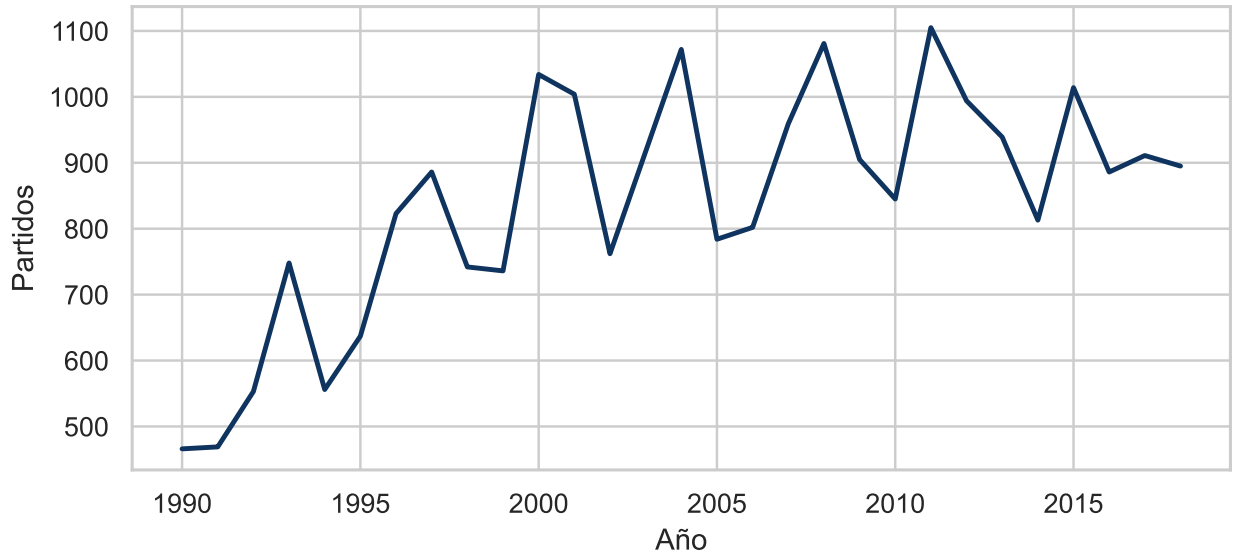


Figura 1: Partidos por año en la muestra de entrenamiento.

El número de partidos por año sube con el tiempo y tiene un dientes de sierra muy marcado: los años de Mundial y de Eurocopa concentran más encuentros que los años intermedios. También se nota el salto de los años noventa, cuando se incorporan las selecciones de los países que surgen de la disolución de la URSS y Yugoslavia.

```
fig, axes = plt.subplots(1, 2, figsize=(9, 3.4), layout="constrained")

sns.countplot(data=train, x="resultado", order=ORDEN, color="#779ab8", ax=axes[0])
axes[0].set(xlabel="Resultado", ylabel="Partidos", title="Todos los partidos")

tabla_sede = (
    train.groupby("sede_neutral")["resultado"]
    .value_counts(normalize=True)
    .rename("proporcion")
    .reset_index()
)
tabla_sede["sede"] = np.where(tabla_sede["sede_neutral"] == 1, "Neutral", "Cancha del local")

sns.barplot(data=tabla_sede, x="resultado", y="proporcion", hue="sede",
            order=ORDEN, ax=axes[1])
axes[1].set(xlabel="Resultado", ylabel="Proporción", title="Según la sede")
plt.show()
```

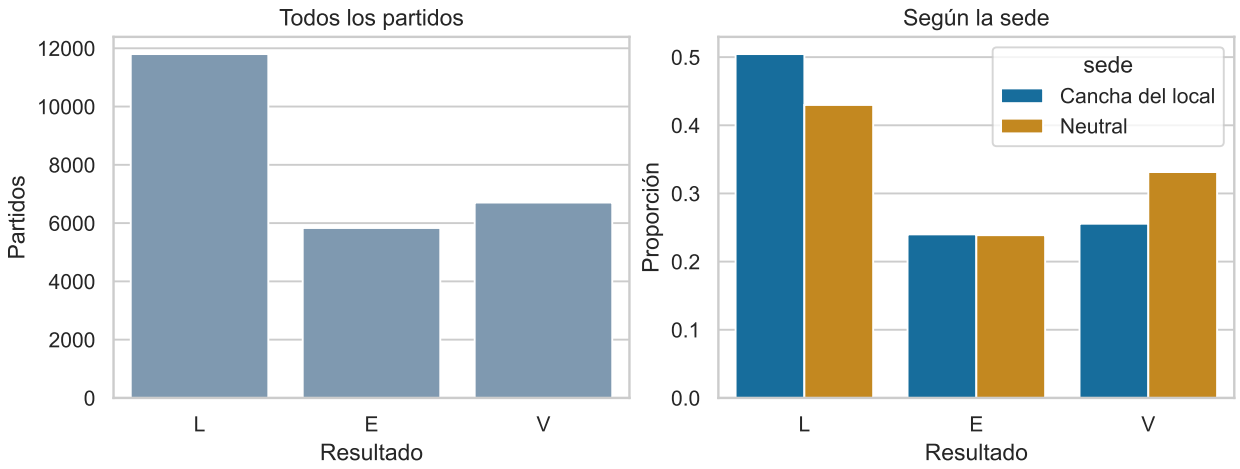


Figura 2: Distribución del resultado, en total y según el tipo de sede.

```
tabla_sede.pivot(index="sede", columns="resultado", values="proporcion")[ORDEN].round(3)
```

resultado sede	L	E	V
Cancha del local	0.504	0.240	0.256
Neutral	0.430	0.239	0.331

Acá aparece el primer hallazgo claro. **En cancha propia el local gana el 50 % de los partidos; en cancha neutral baja al 43 %**, y esos siete puntos se los lleva casi enteros el visitante, que pasa del 26 % al 33 %. La localía existe y se mide.

Vale la pena notar que, aun en sede neutral, el equipo designado como local sigue ganando más que el visitante. Eso no es ventaja de cancha: en un Mundial o una Copa América el "local" de la planilla suele ser el equipo mejor rankeado o el del país anfitrión. Es un recordatorio de que la etiqueta `neutral` no separa dos grupos comparables.

También se ve algo que va a complicarnos el modelado: **el empate nunca es la categoría más frecuente**. Ronda el 24 % y está siempre en el medio, lo que lo hace muy difícil de acertar.

3.2. ¿Cuántos goles se hacen?

```
fig, axes = plt.subplots(1, 2, figsize=(9, 3.4), layout="constrained")

sns.histplot(data=train, x="total_goles", bins=range(0, 13), color=AZUL, ax=axes[0])
axes[0].set(xlabel="Goles en el partido", ylabel="Partidos")

sns.boxplot(data=train, x="es_amistoso", y="total_goles", color="#c5d6e4",
            showfliers=False, ax=axes[1])
```

```
axes[1].set(xlabel="", ylabel="Goles en el partido",
            xticks=[0, 1], xticklabels=["Oficial", "Amistoso"])
plt.show()

print(f"Media:   {train['total_goles'].mean():.2f} goles por partido")
print(f"Mediana: {train['total_goles'].median():.0f}")
print(f"Máximo:  {train['total_goles'].max():.0f}")
```

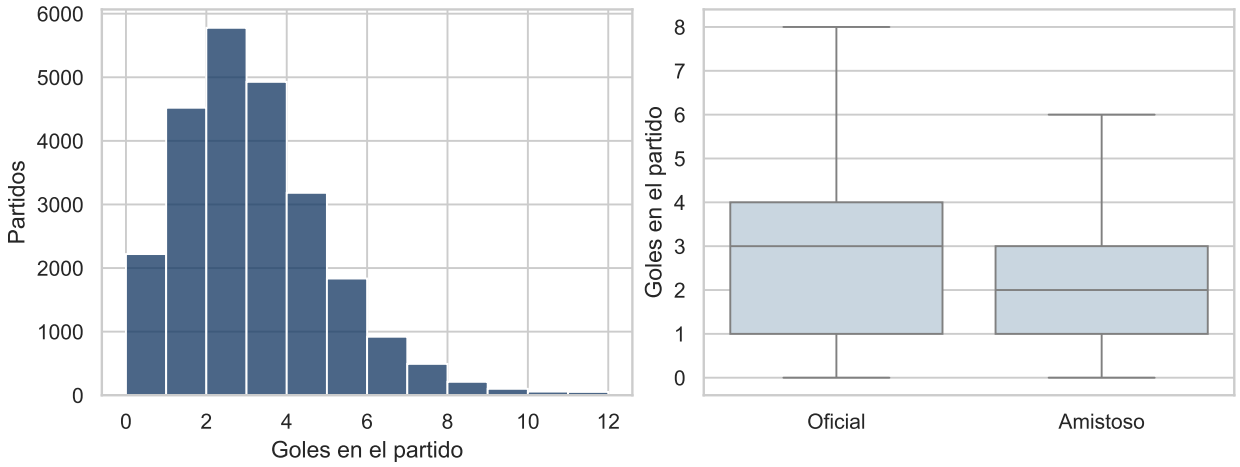


Figura 3: Goles por partido y resultado según el tipo de torneo.

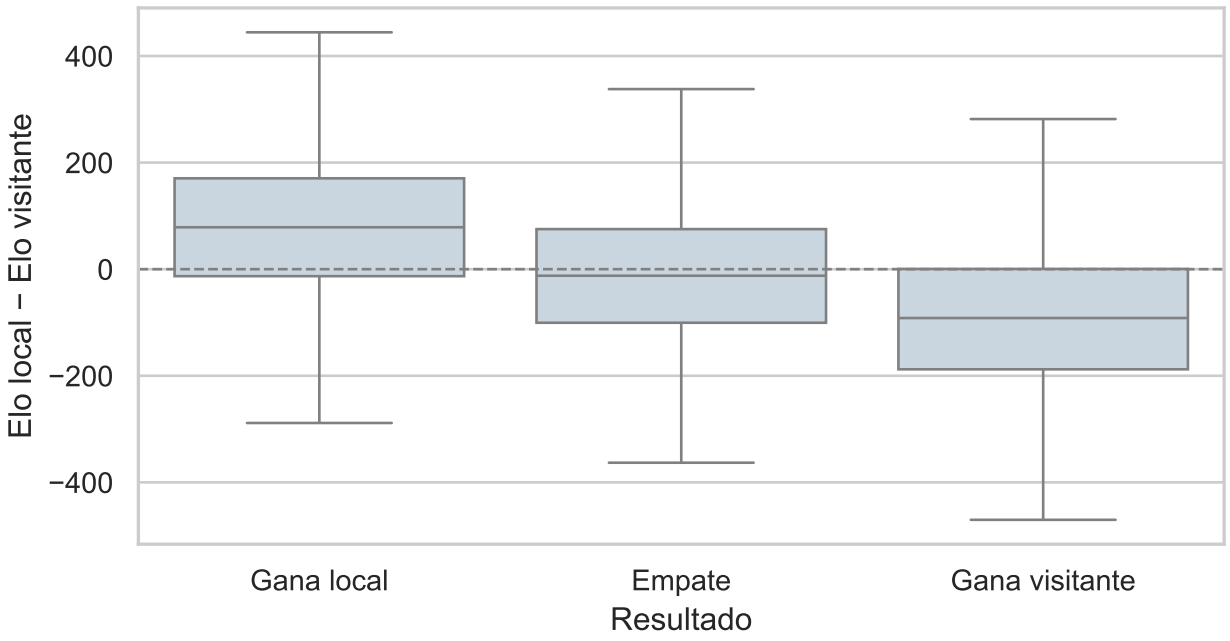
```
Media:   2.72 goles por partido
Mediana: 2
Máximo:  31
```

La distribución está corrida a la derecha: lo más común son 2 o 3 goles, pero hay una cola de goleadas que empuja la media por encima de la mediana. Por eso conviene mirar la distribución entera y no quedarse con el promedio. Amistosos y partidos oficiales tienen cantidades de goles parecidas.

3.3. ¿Sirve el Elo?

```
fig, ax = plt.subplots(figsize=(6.5, 3.4), layout="constrained")
sns.boxplot(data=train, x="resultado", y="elo_dif", order=ORDEN,
            color="#c5d6e4", showfliers=False, ax=ax)
ax.axhline(0, color="gray", linestyle="--", linewidth=1)
ax.set(xlabel="Resultado", ylabel="Elo local - Elo visitante")
ax.set_xticklabels([NOMBRES[r] for r in ORDEN])
plt.show()

train.groupby("resultado")["elo_dif"].mean().reindex(ORDEN).round(1).to_frame("Elo dif. media")
```



(a) Diferencia de Elo antes del partido, según el resultado que terminó dándose.

	Elo dif. media
resultado	
L	83.0
E	-14.3
V	-102.2

(b)

Figura 4

Esta es la figura más importante del análisis exploratorio. **Los partidos que terminan en victoria local son, en promedio, partidos en los que el local ya venía siendo más fuerte**: unos 83 puntos de Elo por encima. En las victorias visitantes la diferencia media es de -102, y los empates quedan justo en el medio, cerca del cero (-14).

Que las cajas se superpongan tanto es también informativo: el Elo ayuda, pero está lejos de determinar el resultado. Hay muchísimos partidos en que gana el más débil.

```
fig, ax = plt.subplots(figsize=(6, 4.5), layout="constrained")
sns.heatmap(train[VARIABLES].corr(), annot=True, fmt=".2f", cmap="vlag",
            vmin=-1, vmax=1, center=0, square=True, ax=ax)
plt.show()
```

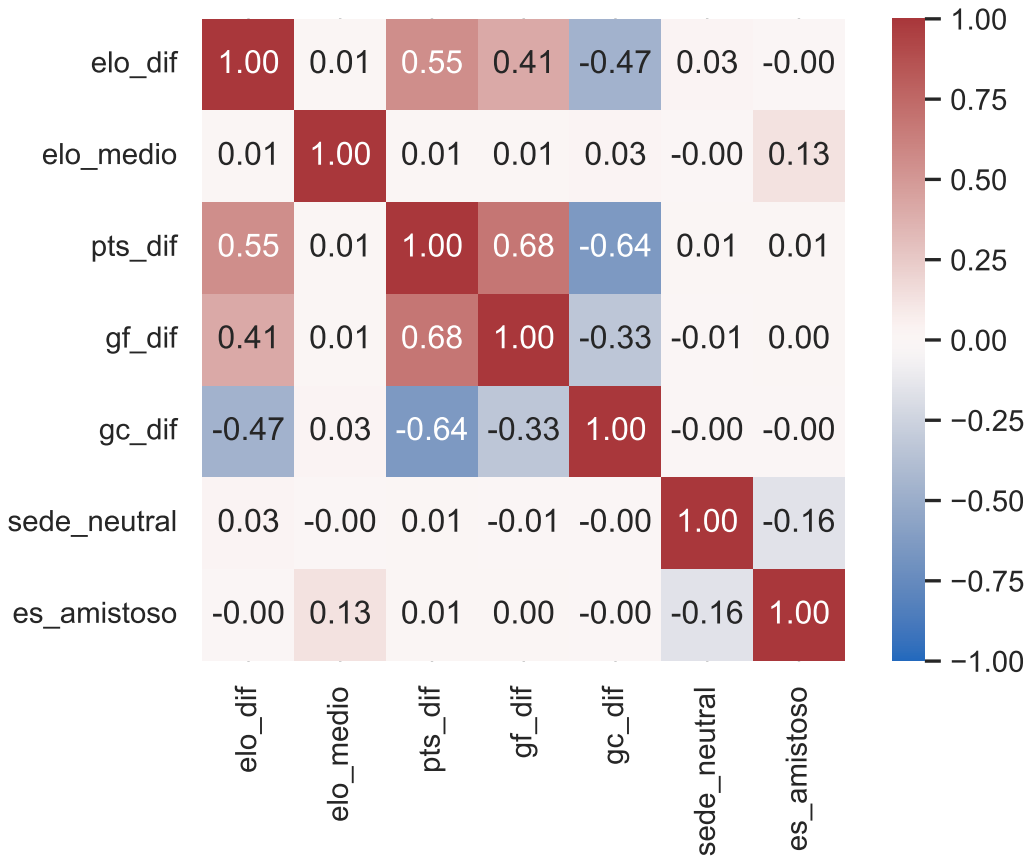


Figura 5: Correlaciones entre las variables construidas.

Las variables de forma reciente están correlacionadas entre sí (quien suma más puntos suele ser quien hace más goles y recibe menos), y todas tienen una correlación moderada con `elo_dif`. Tiene sentido: miden cosas parecidas desde ángulos distintos. No es un problema para un árbol o un random forest, pero sí explica por qué después varias variables aparecen con importancia baja: se pueden sustituir unas por otras.

3.4. Uruguay

```
uy = datos[
    ((datos["home_team"] == "Uruguay") | (datos["away_team"] == "Uruguay"))
    & (datos["date"] >= "1990-01-01")
].copy()

uy["elo_uy"] = np.where(uy["home_team"] == "Uruguay", uy["elo_local"], uy["elo_visita"])

fig, ax = plt.subplots(figsize=(7, 3.2), layout="constrained")
ax.plot(uy["date"], uy["elo_uy"], color=AZUL, linewidth=1.5)
ax.axhline(1500, color="gray", linestyle="--", linewidth=1, label="Promedio inicial")
```

```
ax.set(xlabel="", ylabel="Elo de Uruguay")
ax.legend()
plt.show()
```



Figura 6: Trayectoria del Elo de Uruguay desde 1990.

La curva tiene su punto más bajo a fines de los noventa (1624 puntos de promedio en 1997, los años en que Uruguay no se clasificó a dos Mundiales seguidos) y a partir de ahí sube de forma bastante sostenida: 1717 en 2007, 1767 en 2011 y por encima de 1800 desde 2014. El promedio más alto de toda la serie es el de **2024**, con 1860 puntos.

Conviene leer esto con cuidado. El Elo es acumulativo y no se reinicia nunca, así que una selección que gana de manera consistente durante años va acumulando puntos sin un techo natural. Que el máximo esté al final de la serie no prueba por sí solo que este sea el mejor Uruguay de la historia; dice que viene sumando más de lo que pierde desde hace tiempo.

4. Modelado

4.1. Qué comparamos

La variable a explicar es `resultado` (`L`, `E`, `V`). Comparamos tres opciones, de la más simple a la más compleja:

1. **Referencia:** predecir siempre la categoría más frecuente del entrenamiento. Es el piso que hay que superar; cualquier modelo que no le gane, no sirve.
2. **Árbol de decisión** con profundidad limitada, que tiene la ventaja de poder dibujarse y leerse.
3. **Random forest**, que promedia muchos árboles y suele andar mejor, a costa de ser menos interpretable.

Evaluamos con **accuracy** (proporción de aciertos), la matriz de confusión y el `classification_report`, que muestra precisión y recall categoría por categoría.

```
X_train = train[VARIABLES]
y_train = train["resultado"]
X_test = test[VARIABLES]
y_test = test["resultado"]

print("Entrenamiento:", X_train.shape)
print("Evaluación:   ", X_test.shape)
```

```
Entrenamiento: (24338, 7)
Evaluación:    (6839, 7)
```

4.2. Modelo de referencia

```
clase_mas_comun = y_train.value_counts().idxmax()
acc_referencia = (y_test == clase_mas_comun).mean()

print(f"La categoría más frecuente en entrenamiento es '{clase_mas_comun}' ({NOMBRES[clase_mas_
print(f"Accuracy de predecir siempre '{clase_mas_comun}': {acc_referencia:.3f}")
```

```
La categoría más frecuente en entrenamiento es 'L' (Gana local)
Accuracy de predecir siempre 'L': 0.477
```

4.3. Árbol de decisión

Empezamos con un árbol poco profundo. No es el que mejor anda, pero se puede dibujar y entender de un vistazo qué está mirando.

```
arbol = DecisionTreeClassifier(max_depth=3, random_state=42)
arbol.fit(X_train, y_train)

acc_arbol = arbol.score(X_test, y_test)
print(f"Accuracy del árbol en evaluación: {acc_arbol:.3f}")
```

```
Accuracy del árbol en evaluación: 0.603
```

```
fig, ax = plt.subplots(figsize=(12, 5.5), layout="constrained")
plot_tree(arbol, feature_names=VARIABLES, class_names=arbol.classes_,
          filled=True, rounded=True, fontsize=7, ax=ax)
plt.show()
```

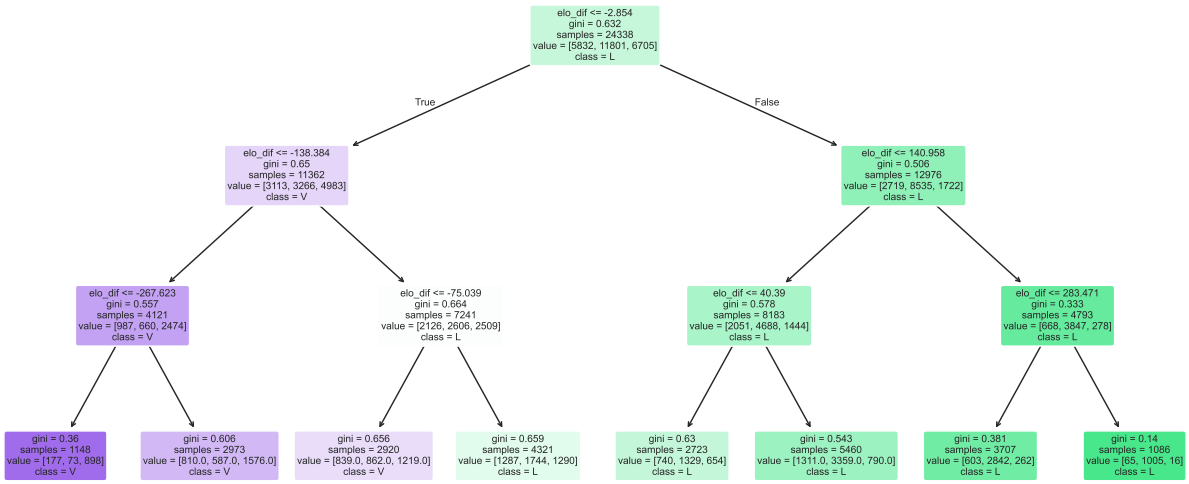


Figura 7: Árbol de decisión de profundidad 3.

El árbol confirma lo del análisis exploratorio, y de una forma más contundente de lo esperado: **los tres primeros cortes los hace por `elo_dif`**. Primero separa en -2.85 (es decir, prácticamente en cero: quién es más fuerte), y después vuelve a cortar la misma variable en -138 y en $+141$. Con tres niveles disponibles, el árbol los gasta enteros en partir la diferencia de Elo en cuatro tramos, sin mirar ninguna otra variable.

Y hay algo todavía más llamativo: **ninguna hoja predice empate**. El árbol nunca apuesta a `E`, porque el empate no es la categoría más probable en ninguno de esos tramos. Volvemos sobre esto al final.

4.4. Random forest

Un solo árbol es inestable: cambiando un poco los datos de entrada, cambia bastante. El random forest entrena muchos árboles sobre muestras distintas y promedia, lo que lo hace más estable.

Elegimos los hiperparámetros con `GridSearchCV`, que prueba todas las combinaciones usando validación cruzada **dentro del entrenamiento**, sin tocar los datos de evaluación.

```
grilla = GridSearchCV(
    RandomForestClassifier(n_estimators=200, random_state=42, n_jobs=-1),
    param_grid={"max_depth": [4, 6, 8, 12], "min_samples_leaf": [5, 20, 50]},
    cv=5,
    n_jobs=-1,
)
grilla.fit(X_train, y_train)

print("Mejores hiperparámetros:", grilla.best_params_)
print(f"Accuracy en validación cruzada: {grilla.best_score_:.3f}")
```

```
Mejores hiperparámetros: {'max_depth': 8, 'min_samples_leaf': 5}
Accuracy en validación cruzada: 0.576
```

```
bosque = grilla.best_estimator_
acc_bosque = bosque.score(X_test, y_test)
print(f"Accuracy del random forest en evaluación: {acc_bosque:.3f}")
```

```
Accuracy del random forest en evaluación: 0.608
```

Comprobamos también qué tan estable es el resultado dentro del entrenamiento, con validación cruzada:

```
puntajes = cross_val_score(bosque, X_train, y_train, cv=5, n_jobs=-1)
print("Accuracy por partición:", puntajes.round(3))
print(f"Promedio: {puntajes.mean():.3f} ± {puntajes.std():.3f}")
```

```
Accuracy por partición: [0.577 0.576 0.576 0.576 0.576]
Promedio: 0.576 ± 0.001
```

La variación entre particiones es chica, así que el número no depende de haber tenido suerte con una división particular.

4.5. Comparación

```
comparacion = pd.DataFrame({
    "Modelo": ["Referencia (siempre local)", "Árbol de decisión", "Random forest"],
    "Accuracy": [acc_referencia, acc_arbol, acc_bosque],
}).round(3)
comparacion
```

	Modelo	Accuracy
0	Referencia (siempre local)	0.477
1	Árbol de decisión	0.603
2	Random forest	0.608

```
fig, ax = plt.subplots(figsize=(6.5, 3), layout="constrained")
sns.barplot(data=comparacion, y="Modelo", x="Accuracy", color="#779ab8", ax=ax)
ax.axvline(acc_referencia, color="#ce5553", linestyle="--", linewidth=1.5)
ax.set(xlabel="Accuracy", ylabel="", xlim=(0, 0.75))
for i, v in enumerate(comparacion["Accuracy"]):
    ax.text(v + 0.01, i, f"{v:.3f}", va="center")
plt.show()
```

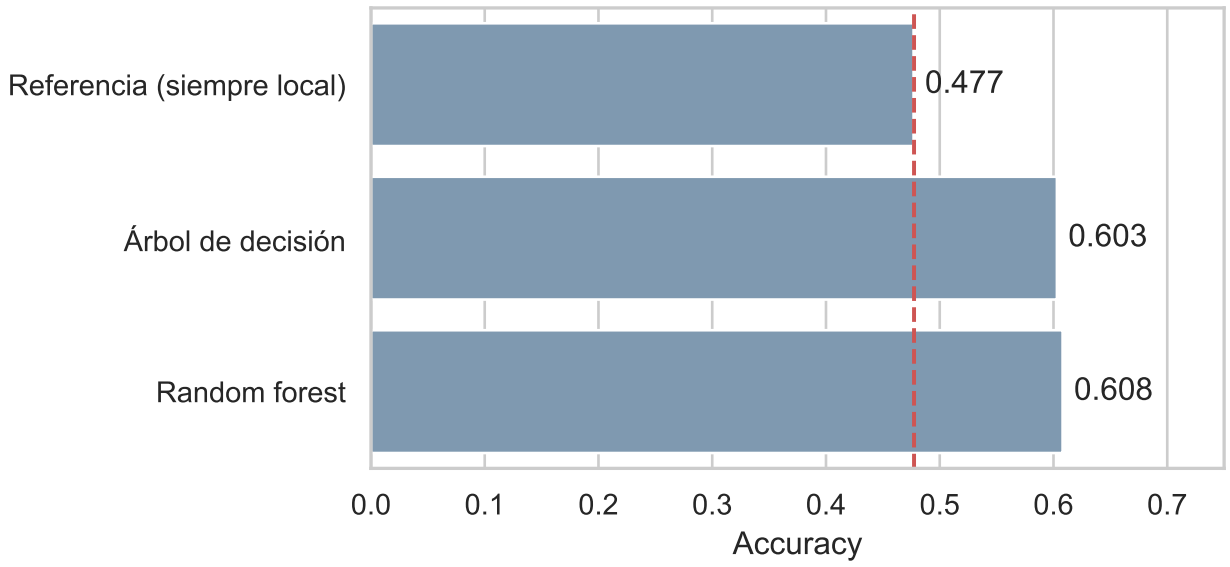


Figura 8: Accuracy de los tres modelos en los datos de evaluación.

Los dos modelos superan a la referencia con holgura: **13 puntos porcentuales** por encima de predecir siempre "gana el local". La información previa al partido **sí aporta**.

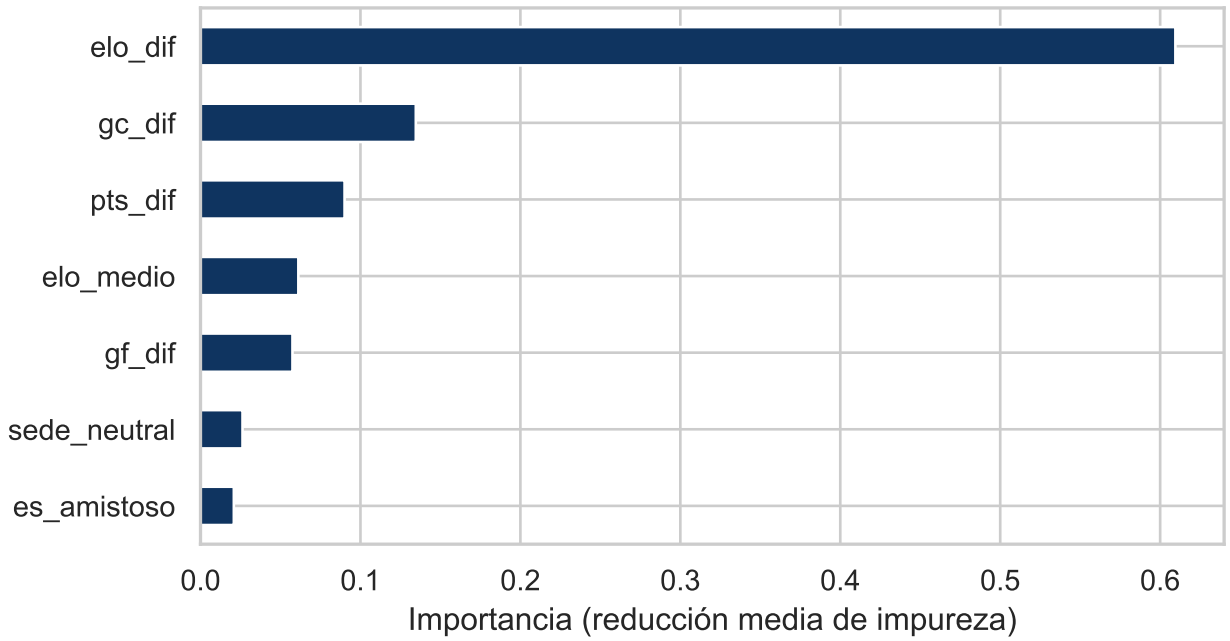
Ahora bien, entre el árbol y el random forest la diferencia es mínima (0.603 contra 0.608). Toda la mejora viene de usar las variables; apenas nada viene de usar un modelo más sofisticado. Es un resultado común y conviene decirlo: acá el trabajo lo hicieron los datos, no el algoritmo.

4.6. Qué variables pesan

```
importancias = pd.Series(bosque.feature_importances_, index=VARIABLES).sort_values()

fig, ax = plt.subplots(figsize=(6.5, 3.4), layout="constrained")
importancias.plot.barh(color=AZUL, ax=ax)
ax.set(xlabel="Importancia (reducción media de impureza)", ylabel="")
plt.show()

importancias.sort_values(ascending=False).round(3).to_frame("importancia")
```



(a) Importancia de cada variable en el random forest.

	importancia
elo_dif	0.610
gc_dif	0.135
pts_dif	0.090
elo_medio	0.061
gf_dif	0.058
sede_neutral	0.026
es_amistoso	0.021

(b)

Figura 9

`elo_dif` domina con holgura: se lleva el 61 % de la importancia total. La fuerza histórica relativa es, de lejos, lo que más informa. Le siguen las variables de forma reciente (`gc_dif` con 0.14 y `pts_dif` con 0.09) y recién después `elo_medio`. `sede_neutral` y `es_amistoso` quedan últimas, con menos del 3 % cada una.

Que `sede_neutral` pese tan poco llama la atención, porque en el análisis exploratorio se veía claramente que la sede importa. La explicación más probable es que el Elo ya absorbe buena parte de ese efecto: como lo calculamos dando ventaja al local en cancha propia, los equipos que juegan mucho de local ya llegan con un Elo que refleja esa condición.

Una advertencia: esta importancia mide cuánto usa el modelo cada variable, no cuánto "causa" cada una. Como varias variables están correlacionadas entre sí, el bosque puede apoyarse en una y dejar las otras con importancia baja aunque contengan información parecida.

4.7. Dónde se equivoca

```
matriz = confusion_matrix(y_test, bosque.predict(X_test), labels=ORDEN)
matriz_prop = matriz / matriz.sum(axis=1, keepdims=True)

fig, ax = plt.subplots(figsize=(5, 4), layout="constrained")
sns.heatmap(matriz_prop, annot=True, fmt=".2f", cmap="Blues", vmin=0, vmax=1,
            xticklabels=[NOMBRES[r] for r in ORDEN],
            yticklabels=[NOMBRES[r] for r in ORDEN], ax=ax)
ax.set(xlabel="Predicho", ylabel="Observado")
plt.show()
```

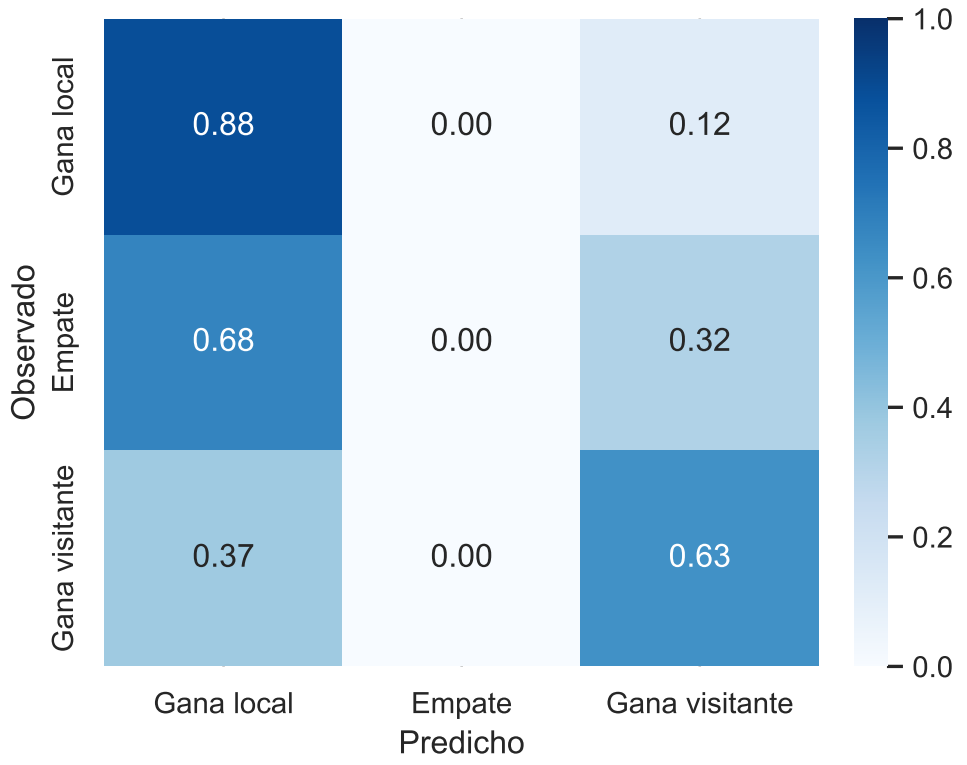


Figura 10: Matriz de confusión del random forest, en proporción de cada fila.

```
print(classification_report(y_test, bosque.predict(X_test),
                           labels=ORDEN,
                           target_names=[NOMBRES[r] for r in ORDEN],
                           zero_division=0))
```

```
                precision    recall  f1-score   support

    Gana local       0.62      0.88      0.73      3265
```

```
        Empate       0.60      0.00      0.01      1555
Gana visitante       0.59      0.63      0.61      2019

      accuracy                           0.61      6839
     macro avg       0.60      0.50      0.45      6839
  weighted avg       0.60      0.61      0.53      6839
```

Acá está la limitación más seria del trabajo, y conviene decirla sin vueltas: **el modelo prácticamente no predice empates.** Reconoce bien las victorias locales, razonablemente las visitantes, y casi ningún empate.

El motivo es estructural. El empate es la categoría menos frecuente y, sobre todo, **nunca es la más probable**: incluso en el partido más parejo imaginable, la probabilidad de empate ronda el 25–30 %, mientras que las de victoria local y visitante se reparten el resto. Como el modelo elige siempre la categoría más probable, el empate nunca gana esa comparación. Es una consecuencia de cómo decide el clasificador, no de que las variables no sirvan.

4.8. Uruguay en los datos de evaluación

```
test_uy = test[(test["home_team"] == "Uruguay") | (test["away_team"] == "Uruguay")].copy()
test_uy["prediccion"] = bosque.predict(test_uy[VARIABLES])

print(f"Partidos de Uruguay entre 2019 y 2025: {len(test_uy)}")
print(f"Accuracy del modelo en esos partidos: {(test_uy['prediccion'] == test_uy['resultado']).
```

```
test_uy[["date", "home_team", "away_team", "home_score", "away_score",
         "resultado", "prediccion"]].tail(10)
```

```
Partidos de Uruguay entre 2019 y 2025: 81
Accuracy del modelo en esos partidos: 0.519
```

	date	home_team	away_team	home_score	away_score	resultado	prediccion
48238	2025-03-21	Uruguay	Argentina	0	1	V	V
48331	2025-03-25	Bolivia	Uruguay	0	0	E	V
48390	2025-06-05	Paraguay	Uruguay	2	0	L	V
48507	2025-06-10	Uruguay	Venezuela	2	0	L	L
48603	2025-09-04	Uruguay	Peru	3	0	L	L
48723	2025-09-09	Chile	Uruguay	0	0	E	V
48802	2025-10-10	Uruguay	Dominican Republic	1	0	L	L
48857	2025-10-13	Uzbekistan	Uruguay	1	2	V	V
48985	2025-11-15	Mexico	Uruguay	0	0	E	L
49020	2025-11-18	United States	Uruguay	5	1	L	V

En los partidos de Uruguay el modelo anda parecido que en el total. Es una muestra chica, así que el número hay que tomarlo como una curiosidad más que como una validación.

5. Comentarios finales

Lo que encontramos. Sí se puede anticipar parcialmente el resultado de un partido de selecciones con información previa. El random forest acierta el 61 % de los partidos contra el 48 % de la regla de referencia: **13 puntos porcentuales de mejora**. La variable que más aporta es, de lejos, la **diferencia de fuerza histórica** medida con el Elo, que concentra el 61 % de la importancia del modelo. La **localía** también importa: en cancha propia el local gana el 50 % de los partidos y en cancha neutral el 43 %. La forma reciente agrega algo, pero bastante menos que el Elo.

Vale la pena subrayar que el árbol de decisión simple acierta casi lo mismo que el random forest (60.3 % contra 60.8 %). La mejora vino de construir buenas variables, no de usar un modelo más complejo.

Lo que no pudimos. El modelo no predice empates. Como el empate nunca es la categoría más probable, un clasificador que elige el máximo nunca lo elige. Esto baja el accuracy global y, sobre todo, significa que el modelo es malo justo en los partidos más parejos, que son los interesantes.

Limitaciones.

- El Elo es nuestro, con parámetros fijos (`K = 20`, ventaja de 60) que no optimizamos. No es el ranking FIFA.
- Los marcadores incluyen alargue pero no penales, así que una final ganada por penales figura como empate.
- La base incluye amistosos y selecciones no afiliadas a FIFA, y la cobertura histórica es despareja.
- Los partidos no son independientes entre sí: las mismas selecciones aparecen muchas veces.
- Dejamos fuera a las selecciones con menos de 10 partidos previos, así que las conclusiones no aplican a equipos nuevos.

Qué se podría hacer después.

- Trabajar con las **probabilidades** (`predict_proba`) en lugar de la categoría predicha. Eso permitiría decir "este partido tiene 30 % de chance de empate" en vez de forzar una única respuesta, que es lo que realmente limita al modelo hoy.
- Probar a bajar el umbral de decisión para el empate, o reagrupar el problema en dos categorías (local gana / no gana).
- Agregar variables que no tenemos: días de descanso, si es partido de eliminatoria o de fase final, distancia de viaje.
- Probar valores de `K` y ventanas de forma distintos, para ver qué tan sensible es el resultado a esas decisiones.

6. Reproducir este trabajo

El repositorio incluye los datos, así que no hay que descargar nada:

```
pip install -r requirements.txt
quarto render analisis_futbol.qmd
```

Eso genera `analisis_futbol.html` y `analisis_futbol.pdf`. También se puede abrir `analisis_futbol.ipynb` en Positron y correr las celdas una por una.

6.1. Referencias

1. Jürisoo, M. *International football results from 1872 to 2026*. Kaggle. Licencia CC0.
2. Materiales del curso *Python para Ciencia de Datos* (2026), clases 1 a 14.
3. Documentación de scikit-learn, pandas y seaborn.